

ANATOMY OF DISTRO

HOW A FROM-SCRATCH GLIBC LINUX IS PUT TOGETHER, PIECE BY PIECE

AUTHOR

Javier Fernández

CONTACT

jfernand@me.com

ISSUED

2026-09-14

ABSTRACT

distro is a Rust CLI that builds a bootable, glibc-based Linux system entirely from upstream source, using nothing but the host's own compiler toolchain. This report explains how the thing is actually put together: what runs first when the machine powers on, what each binary and library in the image is for, how they depend on one another, and how the build pipeline turns seventy-odd separate upstream projects into one disk image. It is organized the way the system itself is layered — kernel, init, a static POSIX base, the seat/session daemons, the Wayland-core libraries, and the graphics stack, up through Weston itself actually rendering a client via virtio-gpu inside QEMU — rather than as a chronological log of work done.

DOCUMENT CONTROL

WORKSPACE	Cargo workspace: builder-core (lib) · distroless (musl/BusyBox) · distro (glibc/from-scratch) · distro-init (PID 1)
TARGET	Native x86_64-unknown-linux-gnu — host toolchain, no cross-compilation
COVERAGE	Everything built and QEMU-verified through Phase 2, plus all of Phase 3: kernel graphics support, libdrm, Mesa, Weston and its cairo/xkeyboard-config chain, actually running with a client connected and rendering via virtio-gpu inside QEMU — see §7. Also: every one of distro's 25 packages is now a Buildpack — the old fetch.rs/userland.rs are gone entirely — see §8.5
NOT COVERED	Phase 4 through Phase 6 — a Rust toolchain on-target, COSMIC itself, real GPU drivers beyond virtio-gpu, audio, networking UI — see §9

01

WHAT DISTRO IS

Most ways to get a custom Linux system booting quickly start from someone else’s finished distribution — `debootstrap`, a container base image, an Arch `pacstrap` — and layer changes on top. This project rejected that: every binary in the built image is compiled here, from source, against nothing but the host machine’s own `glibc` and `gcc`. The one exception is the same one every real distribution makes: the host’s **compiler toolchain** — `gcc`, `binutils`, `meson`, `autoconf` — is infrastructure, not distro content, the same way a bootstrap compiler is infrastructure for a self-hosting language.

CRATE	ROLE
<code>builder-core</code>	Shared library: kernel build (<code>FEATURE_PACKS</code>), disk-image assembly, the QEMU test harness, the USB writer. Used by both distros unchanged.
<code>distroless</code>	A separate, minimal distro built by this same workspace: <code>musl</code> + <code>BusyBox</code> + <code>utils</code> , cross-compiled. Not covered here.
<code>distro</code>	The subject of this report: a from-scratch <code>glibc</code> system, native-compiled, aimed eventually at a COSMIC desktop.
<code>distro-init</code>	A from-scratch PID 1 written for this project, ~100 lines of Rust — see §2.2.

- § 2
- The boot sequence — what actually happens, in order, from power-on to a shell.
- § 3
- The kernel — the config baseline, the feature packs, and how to pick a version.
- § 4
- The static base: `coreutils`, `shell`, `login` — one binary in, no shared-library bookkeeping.
- § 5
- The seat/session layer: `seatd`, `dbus`, `eudev` — what each one actually does.
- § 6
- The Wayland-core libraries — built, installed, not yet used by anything.
- § 7
- The graphics stack: `libdrm`, `Mesa`, what DRI/Gallium/EGL/GBM actually are, and Weston’s own cairo dependency chain.
- § 8
- How it’s all actually built: the pipeline, the config file, the `sysroot`, and a new buildpack architecture in progress.
- § 9
- What isn’t part of the picture yet.

02

THE BOOT SEQUENCE

The clearest way to see how the pieces fit is to follow what actually happens, in order, when the built image boots. At a high level, every Linux system does the same three things: a **bootloader** (GRUB) finds and loads the **kernel**; the kernel initializes hardware, mounts the root filesystem, and executes exactly one program as process ID 1 — conventionally called **init**; and everything else on the system is, directly or indirectly, started by that init process. What differs system to system is entirely what init actually does next — `systemd` starts hundreds of units in dependency order; `BusyBox`’s init reads a small `/etc/inittab`; this project writes its own init (§2.2) that does five things and nothing else.

STEP	COMPONENT	WHAT HAPPENS
1	GRUB	Reads <code>/boot/vmlinuz</code> and hands off. Installed by <code>builder-core</code> ’s image stage, unchanged from <code>distroless</code> .
2	Kernel	Boots, mounts the <code>ext4</code> rootfs read-write, execs <code>/sbin/init</code> . Built by <code>builder-core::stages::kernel</code> — the same generic stage <code>distroless</code> uses.
3	<code>distro-init</code>	Our own PID 1 (§2.2). Mounts <code>/proc</code> , <code>/sys</code> , <code>/dev</code> (<code>devtmpfs</code>), and a fresh <code>tmpfs</code> at <code>/run</code> .
4	<code>udev</code> , <code>seatd</code> , <code>dbus-daemon</code>	Forked and exec’d, in that order. <code>distro-init</code> then runs <code>udevadm trigger</code> once — a coldplug that tells <code>udev</code> about the devices <code>devtmpfs</code> already created before it started.
5	<code>agetty</code> ×2	Forked on <code>tty1</code> (VGA console) and <code>ttyS0</code> (serial, for QEMU/headless use). Whichever exits gets respawned.

STEP	COMPONENT	WHAT HAPPENS
6	login	<code>agetty</code> execs <code>/bin/login</code> once a username is typed. <code>login</code> reads <code>/etc/passwd</code> + <code>/etc/shadow</code> for real — the <code>root</code> account's shadow entry has an empty password field, so <code>login</code> skips the password prompt entirely rather than reimplementing BusyBox's separate no-password shortcut.
7	bash	<code>login</code> execs the shell named in <code>/etc/passwd</code> . A real, interactive shell — not a script, not a fallback.

THIS IS NOT A DIAGRAM — IT WAS WATCHED HAPPENING

Every arrow above was reproduced live in QEMU with a scripted serial-console session, not inferred from source reading: `login: root` → no password prompt → `-bash-5.2#` → `dbus-send --system ... ListNames` returns a real reply → `udevadm info --query=all --name=/dev/tty1` returns a populated device entry. See §8.4 for the harness.

2.1 WHY A CUSTOM INIT AT ALL

BusyBox (which `distroless` uses) ships its own tiny init built in; nothing glibc-based has an equivalent single binary. Rather than reach for `systemd` — which would drag in `logind`, `udev`, `journald`, and a great deal more than this system currently needs — `distro-init` is ~100 lines of Rust using the `nix` crate:

DISTRO-INIT/SRC/MAIN.RS — THE WHOLE SUPERVISION LOOP, ABBREVIATED

```
fn main() {
    mount_basic_filesystems();           // proc, sysfs, devtmpfs, tmpfs at /run

    let mut udevd_pid = spawn_udevd();   // no -d: stays a tracked child, not a daemon
    let mut seatd_pid = spawn_seatd();
    let mut dbus_pid = spawn_dbus();     // --nofork, same reasoning
    coldplug_devices();                  // `udevadm trigger`, one-shot

    let mut tty1_pid = spawn_tty1();     // agetty
    let mut serial_pid = spawn_serial();

    loop {
        match waitpid(None, Some(WaitPidFlag::empty())) {
            Ok(WaitStatus::Exited(pid, _)) | Ok(WaitStatus::Signaled(pid, _, _)) => {
                // whichever of the five died gets forked again; anything
                // else reaped here was just an orphaned grandchild.
            }
            ...
        }
    }
}
```

Every daemon it starts would normally daemonize itself (fork, detach, exit the parent) — `dbus-daemon --nofork` and `udev` with `no -d` suppress that, so they stay `distro-init`'s direct children and its `waitpid` loop actually sees them exit if they crash.

03 THE KERNEL

Step 2 of §2's boot table — the kernel itself — is built by `builder-core::stages::kernel`, the same code `distroless` uses unchanged. `distro`'s own `distro.toml` names a version and a download URL (currently 6.17, from kernel.org) under `[kernel]`; `distro build-kernel` fetches, configures, and compiles it.

3.1 WHERE THE STARTING CONFIG COMES FROM

There are two starting points, chosen by whether `kernel.config_file` is set in the config file:

- **No `config_file` (the default, and what `distro` currently uses).** Runs `make defconfig` — the kernel's own “reasonable defaults for this architecture” config — and then strips it down: every option belonging to one of the named feature packs below gets turned off, regardless of whether `defconfig` turned it

on. Everything `defconfig` sets that **isn't** owned by a pack (PCI, ACPI, EFI boot, block/ATA/virtio, ext4/vfat, the console) is left exactly as `defconfig` set it — only the packs are deliberately stripped, not the boot-essential baseline.

- **config_file set.** A previously saved `.config` (produced by an interactive `make menuconfig` session — see below) is used as-is, only re-resolved via `make olddefconfig` in case it predates a kernel upgrade. Nothing gets stripped: whoever hand-picked that config already decided what they wanted.

Either way, whatever packs are named in `kernel.features` get turned back on as the final step — so even a hand-edited `config_file` can still request named packs on top of it, without needing to know the underlying `CONFIG_*` symbols.

3.2 THE FEATURE PACKS

Fourteen named bundles, each just a curated list of Kconfig options one `kernel.features` entry away from being switched on as a unit:

PACK	WHAT IT TURNS ON
graphics	DRM/KMS graphics + fbdev console (i915, virtio-gpu, bochs, AGP) instead of plain VGA text.
sound	ALSA sound subsystem and the Intel HDA driver.
wireless	The Wi-Fi stack (cfg80211/mac80211) and rkill.
hid-extras	Per-vendor HID quirk drivers (Sony, Samsung, Gyration, ...) and the hiddev/hidraw userspace interfaces — generic USB HID keyboards/mice work without this.
legacy-nics	Dedicated Ethernet chipset drivers (Tigon3, Tulip, E100/E1000(E), Sky2, Forcedeth, 8139too, R8169) for real hardware — QEMU's virtio-net always works without this.
legacy-buses	PCMCIA/CardBus (Yenta) and legacy PATA chipset drivers — AHCI/virtio-blk always work without this.
network-fs	NFS (client + root-over-NFS), 9P, and autofs.
netfilter	Connection tracking, NAT, and iptables — only useful if this box routes or firewalls traffic.
security-extras	Disk quotas, POSIX ACLs, and SELinux.
iommu	AMD/Intel IOMMU support — only needed for PCI passthrough or running this as a virtualization host.
debug	Kernel debug instrumentation (Magic SysRq, schedstats, block-IO tracing, boot-param/entry debug) — useful while bringing up boot, dead weight once stable.
ia32-emulation	Run 32-bit x86 binaries on this 64-bit kernel.
iso9660	ISO9660/Joliet/zisofs filesystem support, for booting or mounting optical media images.
boot-logo	Framebuffer console + boot-time Linux logo (the stock penguin, or a custom image via <code>kernel.logo_file</code> — must be an 80×80, ASCII/P3 PPM with at most 224 colors, the kernel's own logo converter's hard requirement).

DISTRO'S CURRENT KERNEL.FEATURES: NONE

`distro.toml` sets no `kernel.features` at all — every pack above is off, which is why `distro`'s kernel today is the fully stripped `defconfig` baseline. Phase 3 (Mesa/graphics) will need at least `graphics` turned on; the original project roadmap expects most of these packs on for a real desktop eventually, none of that has happened yet.

3.3 MENUCONFIG'S OWN MENU TREE, AND WHAT WE ACTUALLY TOUCH

The fourteen packs above are this project's own grouping, not the kernel's. Run `make menuconfig` against the fetched sources (via `menu-config`, §3.4) and the kernel presents its **own** top-level menu structure — fourteen menus of its own, as it happens, though not the same fourteen. Reading straight from this kernel's own Kconfig files (not from memory), in the order the config parser actually reaches them:

MENUCONFIG'S TOP-LEVEL MENU	DO ANY OF OUR PACKS TOUCH IT?
General setup	No — left exactly as <code>defconfig</code> set it. Init system choice, cgroups, namespaces, <code>printk</code> , the core boot-essential baseline.
Processor type and features	No — CPU family, SMP, NUMA, <code>IA32_EMULATION</code> 's sibling 64-bit options all stay at <code>defconfig</code> defaults.
Power management and ACPI options	No — ACPI itself is part of the untouched boot-essential baseline the project's own docs call out explicitly.
Bus options (PCI etc.)	Yes — <code>legacy-buses</code> (PCMCIA/CardBus). Plain PCI itself is untouched.
Binary Emulations	Yes — <code>ia32-emulation</code> is literally the only option in this menu our packs name.
Executable file formats	No — ELF support stays at <code>defconfig</code> defaults.
Memory Management options	No — untouched.
Networking support	Yes — <code>wireless</code> (<code>cfg80211/mac80211/rfkill</code>) and <code>netfilter</code> (<code>conntrack/NAT/iptables</code>) both live here.
Device Drivers	Yes, the most — <code>graphics</code> , <code>sound</code> , <code>wireless</code> (the actual wireless-LAN drivers, as opposed to the stack above), <code>hid-extras</code> , <code>legacy-nics</code> , <code>legacy-buses</code> (PATA chipset drivers), <code>boot-logo</code> , and <code>iommu</code> all touch submenus here.
File systems	Yes — <code>network-fs</code> (NFS/9P/autofs), <code>iso9660</code> , and part of <code>security-extras</code> (disk quotas).
Security options	Yes — <code>security-extras</code> (SELinux).
Cryptographic API	No — untouched.
Library routines	No — untouched.
Kernel hacking	Yes — <code>debug</code> .

READING THIS THE OTHER WAY ROUND

Seven of the kernel's fourteen top-level menus have at least one pack reaching into them; the other seven — **General setup**, **Processor type and features**, **Power management and ACPI options**, **Executable file formats**, **Memory Management options**, **Cryptographic API**, and **Library routines** — the boot-essential core, plus crypto and the C library shims — are entirely untouched by this project's own config, left exactly as `defconfig` produced them. That's deliberate: the packs exist to strip **optional** hardware/feature surface, not to second-guess what a working x86_64 boot actually requires.

3.4 WHAT'S EXPOSED BY EACH CLI

Everything in §3.1–3.2 lives in `builder-core::stages::kernel` as generic, shared code — both distros' kernels are configured by the same functions. What differs is how much of it each CLI actually puts a command in front of, versus leaving as “edit the TOML file yourself”:

CAPABILITY	DISTROLESS	DISTRO
Fetch kernel source	<code>fetch [--clean]</code>	<code>fetch</code>
Build the kernel	<code>build-kernel</code>	<code>build-kernel</code>
Pick a version from kernel.org	<code>resolve-kernel --channel <stable\ lts></code>	<code>resolve-kernel --channel <stable\ lts></code>
Interactive <code>make menuconfig</code>	<code>menu-config --save-to <path></code>	<code>menu-config --save-to <path></code>
List the available feature packs	<code>list-features</code>	<code>list-features</code>

CAPABILITY	DISTROLESS	DISTRO
Turn feature packs on	<code>kernel.features = [...]</code> in the config file	same: <code>kernel.features = [...]</code> in <code>distro.toml</code> — config-file level support is identical
Custom boot logo	<code>kernel.logo_file</code> in the config file	same, <code>kernel.logo_file</code> in <code>distro.toml</code>
Force any stage to rerun	global <code>--force</code> flag	global <code>--force</code> flag
Interactive dashboard	<code>tui</code> — a full terminal UI for every stage plus USB writing	no equivalent

`resolve-kernel`, `menu-config`, and `list-features` are now wired up on both sides. `menu-config` reuses `builder-core`'s `menuconfig()` directly (it only reads `cfg.kernel_build_dir()` and writes the saved config to a separate `--save-to` file, never touching the distro's own config file, so it's safe unmodified against either `Config` type), and `list-features` reuses the same `println!` loop over `FEATURE_PACKS` `distroless`'s own `list_features()` uses. `resolve-kernel` needed its own thin wrapper in `distro/src/stages/kernel.rs`: `builder-core`'s version loads and saves a whole `builder_core::config::Config`, which doesn't have `distro`'s `bash / util_linux / shadow / seatd / ...` sections — loading `distro.toml` through it would fail to parse, and saving would silently drop everything those functions don't know about. The `kernel.org` lookup itself was pulled out into a shared `latest_release()` so both wrappers reuse the same HTTP/JSON logic without duplicating it.

The only remaining CLI-surface difference in this table is `distroless`'s `tui` — no equivalent exists for `distro`, and nothing in this section claims otherwise.

04

THE STATIC BASE

Five packages exist purely to get from a mounted rootfs to a real, authenticated shell.

COREUTILS, BUSYBOX, AND UTILS — THREE ANSWERS TO THE SAME QUESTION

Every Unix system needs a basic set of file and text commands — `ls`, `cat`, `cp`, `mv`, `rm`, and so on. “Coreutils” is the generic name for that toolset, not one specific program. **GNU coreutils** is the implementation most desktop Linux distributions ship. **BusyBox** is a different, much older answer aimed at tiny/embedded systems: a single small C binary that crams coreutils **and** a shell, `init`, `mount`, `ps`, and dozens of other tools into one multi-call executable — this workspace's other distro, `distroless`, uses it. **utils** is a third answer: a from-scratch reimplementa-tion of just coreutils, in Rust. This project uses `utils` instead of `BusyBox` for `distro` specifically because of the “prefer Rust alternatives to traditional tools” goal behind the whole `distro` crate — `BusyBox` would have worked technically, the same way it does for `distroless`.

The `glibc` equivalent of what `BusyBox` does as one binary is spread across five separate upstream projects here, because `glibc-land` has no single equivalent to reach for.

PACKAGE	VERSION	PROVIDES	WHY IT'S THERE
<code>utils/coreutils</code>	<code>git main</code>	<code>ls</code> , <code>cat</code> , <code>cp</code> , ...	A Rust reimplementa-tion of GNU coreutils — one multi-call binary, symlinked under every applet name. The built feature set is a curated subset (see §9), not the full set.
<code>bash</code>	5.2.37	<code>/bin/bash</code> , <code>/bin/sh</code>	The login shell named in <code>/etc/passwd</code> .
<code>util-linux</code>	2.41.2	<code>agetty</code> , <code>mount</code> , <code>umount</code>	Built with <code>--disable-all-programs</code> plus explicit <code>--enable-*</code> for just these three — <code>util-linux</code> ships dozens of tools, only these are needed.

PACKAGE	VERSION	PROVIDES	WHY IT'S THERE
shadow-utils	4.17.4	login, passwd	Real /etc/passwd + /etc/shadow authentication — not BusyBox's separate empty-password mechanism, genuine shadow-file semantics.

All five are **statically linked** — `--disable-shared --enable-static` at configure time, `LDFLAGS=-all-static` at make time (plain `-static` alone breaks configure's own compiler sanity check once libtool is involved). One binary compiled, one binary copied into the rootfs, nothing else to track. This is why they sit apart from everything in §5–7: dynamic linking doesn't enter the picture until `dbus` (§5), which needs `libexpat` and isn't meaningfully staticable.

A DISPATCH BUG SPECIFIC TO THIS BUILD

`utils` decides which applet `argv[0]` refers to by reading the kernel's `AT_EXECFN` auxval on non-musl Linux (hardening against `argv[0]` spoofing) instead of trusting `argv[0]` directly. On this build host — and inside this project's own kernel — `AT_EXECFN` comes back **empty**, so every applet invocation hit "<unknown binary name>" before dispatch even started. `distro/src/stages/fetch.rs` patches a fallback to `argv0` into `utils'` `src/common/validation.rs` on every fetch, idempotently.

05 THE SEAT & SESSION LAYER

Three daemons exist to do the things a desktop session needs **before** any compositor can run: let an unprivileged process touch `/dev/input` and `/dev/dri`, pass messages between processes, and know what hardware is plugged in. None of them are optional for Phase 3 — they're the floor a compositor stands on, not decoration.

WHAT UDEV ACTUALLY IS

The kernel's device drivers know about hardware, but they don't manage `/dev` themselves in any structured way — `devtmpfs` (already mounted by `distro-init`, §2) creates basic device nodes automatically as drivers load, and that's all it does. `udev` is the userspace daemon that listens for the kernel's own hardware-change announcements (sent over a netlink socket whenever something is plugged in, unplugged, or otherwise changes state), and in response creates or removes the matching `/dev` entries, applies permissions/rules, and maintains a live, queryable database of what hardware currently exists — the thing `udevadm info` reads from. Without it running, nothing gets notified when a device appears after boot, and nothing can ask "what's connected right now" in a structured way. `eudev` is a fork of `udev` that works without `systemd`, which this project needs since it uses `seatd` instead of `systemd-logind` (below) — ordinary `udev` is a `systemd` subproject these days.

PACKAGE	VERSION	WHAT IT ACTUALLY DOES
seatd	0.9.3	Owns <code>/dev/input/*</code> and <code>/dev/dri/*</code> on behalf of whatever process asks nicely over its socket, so a compositor doesn't need to run as root to touch a keyboard or a GPU. Its own README says it "depends only on libc" — the <code>systemd-logind</code> alternative, chosen explicitly over pulling in <code>systemd</code> (see the callout below).
dbus	1.16.2	The system message bus every desktop component uses to talk to every other one. Runs as root here (<code>-Ddbus_user=root</code>) — the rootfs has no unprivileged <code>messagebus</code> user yet to drop privileges to.
eudev	3.2.14	A <code>systemd</code> -independent fork of <code>udev</code> (what Alpine, Void, and Gentoo use without <code>systemd</code>) — walks <code>/sys</code> , builds a device database, and exposes it as <code>libudev</code> . Exists in this pipeline for one specific reason: <code>libinput</code> (§6) hard-depends on <code>libudev</code> , and there is no way around that dependency.

SEATD INSTEAD OF SYSTEMD – AN OPEN QUESTION, NOT A CLOSED ONE

Choosing `seatd` over `systemd` was a deliberate decision, made explicitly rather than defaulted into, to avoid pulling in `logind`, `full udev`, `journald`, and `networkd` for one seat-management socket. Whether `seatd` alone will actually satisfy COSMIC once Phase 5 needs a real session is still an open question — this system proves `seatd` starts and holds a seat, not that COSMIC will accept it as `logind`'s replacement.

`distro-init` starts all three (plus `udev`, which is really this same device-management job — see §2.1's table) and supervises them the same way it supervises `agetty`. `dbus`'s socket lands at `/run/dbus/system_bus_socket` and `distro-init` creates `/run/dbus` itself immediately after mounting a fresh `tmpfs` at `/run`, since nothing else would.

06

THE WAYLAND-CORE LIBRARIES

Nothing in this section runs. Seven libraries are built and installed into the rootfs, waiting for Phase 3's compositor to be the first thing that actually links against any of them — verification at this layer is “builds and installs cleanly against everything before it,” not “does something observable.”

PACKAGE	VERSION	WHAT IT'S FOR
<code>wayland</code>	1.26.0	The core wire-protocol libraries (client, server, cursor, EGL) and <code>wayland-scanner</code> , the code generator every later Wayland package runs at its own build time.
<code>wayland-protocols</code>	1.49	The actual protocol definitions — <code>xdg-shell</code> and the rest — as XML. No library of its own, just data plus a <code>pkg-config</code> file.
<code>libxkbcommon</code>	1.12.4	Turns “us, evdev, pc105” into the keymap tables a compositor hands to clients. X11 support is off (no <code>libxcb</code> — see §9); real keymap compilation needs the <code>xkeyboard-config</code> data package, also not built yet.
<code>pixman</code>	0.46.4	Software rasterization — Mesa's fallback path, and some compositor-side operations not worth sending to the GPU.
<code>libdisplay-info</code>	0.4.0	Parses a monitor's own EDID/DisplayID — how a compositor learns what resolutions and refresh rates a display actually supports.
<code>libevdev</code>	1.13.7	Reads and writes raw evdev input-device events. <code>libinput</code> 's one mandatory dependency.
<code>libinput</code>	1.31.3	Turns raw evdev events into the pointer/keyboard/touch/gesture events a compositor actually wants. <code>libwacom</code> (tablets) and <code>mtdev</code> (legacy multitouch) are both left out — see §9.

The dependency order between them is also the build order: `wayland` first (nothing else here can build without `wayland-scanner`), then `wayland-protocols` (needs `wayland-scanner` on `PATH`), then the rest, with `libinput` last since it needs both `evdev`'s `libudev` (§5) and `libevdev`.

07

THE GRAPHICS STACK

Phase 3's charter, per the project's own roadmap: get a graphics stack working, scoped first to QEMU's own virtual GPU rather than real hardware. Nothing in this section produces pixels on screen yet either — the roadmap's own Phase 3 milestone is a minimal Wayland client actually rendering something via `virtio-gpu`, which needs a compositor to host it and is still ahead, not part of what's covered here — but this is the piece that turns “a kernel that can talk to a GPU” into “a library a compositor can actually call into.”

DRI, GALLIUM, EGL, GBM — WHAT THESE ACTUALLY ARE

The kernel's DRM subsystem (already built — `CONFIG_DRM_VIRTIO_GPU`, part of the `graphics` feature pack, §3.2) owns the GPU at the lowest level: it hands out framebuffers and submits command buffers, but has no idea what “draw a triangle” means. **Mesa** is the userspace library that turns OpenGL/OpenGL ES calls into whatever that specific GPU actually understands. **Gallium** is Mesa's own internal plumbing for doing that once per GPU family instead of once per API — a “Gallium driver” is the translator for one specific GPU (or, here, one specific **virtual** one). **DRI** (Direct Rendering Infrastructure) is the convention by which an application actually finds and loads the right one at runtime. **EGL** is the glue between a window system (Wayland, here) and an OpenGL/GLES context — it's what a compositor or client actually links against, not Mesa's internals directly. **GBM** (Generic Buffer Management) is how a Wayland compositor allocates the actual pixel buffers it hands to the DRM/KMS display hardware — the piece that makes a rendered frame actually show up on a screen, as opposed to just existing in GPU memory.

PACKAGE	VERSION	WHAT IT'S FOR
libdrm	2.4.134	The kernel-userspace ioctl wrapper every GPU-facing library, Mesa included, builds on. Every vendor-specific sub-library (Intel/AMD/nouveau/...) is disabled — virtio-gpu needs only libdrm's generic core.
Mesa	26.2.2	<code>libEGL</code> , <code>libGLv2</code> , <code>libgbm</code> , and the actual Gallium driver (<code>libgallium-26.2.2.so</code>) — built scoped to just the <code>virgl</code> (talks to QEMU's virtio-gpu/virgl backend, hardware-accelerated) and <code>softpipe</code> (software fallback) Gallium drivers.

Mesa's build answers a question the roadmap explicitly left open: whether its virtio-gpu path needs LLVM (for `llvmpipe`, the LLVM-based software rasterizer) or can go LLVM-free. It can — `llvmpipe` isn't in the driver list above, `-Dllvm=disabled` is passed explicitly, and Mesa built and installed cleanly regardless, since neither `virgl` nor `softpipe` needs LLVM for anything. Vulkan is off entirely (`-Dvulkan-drivers=`, empty — this milestone is OpenGL/GLES only), and so is GLX/X11 (`-Dplatforms=wayland`, `-Dglx=disabled`), consistent with the earlier decision not to build `libxcb` (§9).

A TOOLCHAIN BUMP: MESON VIA PIP, NOT APT

Mesa needs meson ≥ 1.4.0; Ubuntu 24.04's own `apt` package is 1.3.2. `build-toolchain` now also checks the installed meson's version and, if it's too old, runs `pip install --user --upgrade meson` — `sysroot_env` puts `~/.local/bin` ahead of `/usr/bin` on `PATH` explicitly for every build invocation from here on, rather than assuming the invoking shell already has it there.

ONE DESIGN LIMITATION, THREE DIFFERENT PACKAGES

PKG_CONFIG_SYSROOT_DIR (§8.3) is essential for letting one sysroot package’s build find another’s `-I / -L` flags correctly — but it applies uniformly to **everything** pkg-config resolves, with no way to tell “a package genuinely installed under our sysroot” apart from “a host build tool that happens to be installed for unrelated reasons.” Phase 3 hit this three separate times, needing three different fixes — the same underlying limitation surfacing again, not the same bug recurring:

- **Mesa’s optional spirv-tools support** found this build host’s own Homebrew-installed SPIRV-Tools (a nonstandard prefix, `/home/linuxbrew/...`), whose real include path then got rewritten into a sysroot location it was never installed under. Fixed by forcing `PKG_CONFIG=/usr/bin/pkg-config` in `sysroot_env` — the system pkg-config’s own default search dirs never point outside a normal Ubuntu install, unlike Homebrew’s own pkg-config wrapper, which bakes in Homebrew’s paths as compiled-in defaults.
- **libxkbcommon’s optional xkbregistry** (XDG-style layout enumeration — not needed for keymap compilation itself) needed `libxml2`, which turned out to be genuinely installed at the **standard** system location too (`apt` ’s `libxml2-dev`) — the `PKG_CONFIG` fix above doesn’t help when the leak is a real system package, not a Homebrew one. Fixed by disabling the feature outright (`-Denable-xkbregistry=false`), since it isn’t needed anyway.
- **libdisplay-info’s hwdata lookup** (a vendor-ID database it embeds at **build** time, nothing reads it at target runtime) isn’t optional the way the other two are, so neither fix above applied. Its own `meson.build` already had a literal, correct fallback path for exactly this case (`/usr/share/hwdata/pnp.ids`) — unused because the `dependency('hwdata')` lookup “succeeds” first, then gets the same sysroot-mangling treatment. `distro/src/stages/fetch.rs` now patches this file on every fetch (idempotently, same pattern as the existing `utils AT_EXECPN patch, §4`) to always take that branch.

A fourth, unrelated bug surfaced in the same round of testing: `meson setup` refuses to reconfigure an already-configured `build/` directory, and fails outright — not just warns — if that directory was last configured by an older meson than is now on `PATH` (exactly what happened switching to pip’s meson mid-project). `meson_build_and_install` now removes any existing `build/` before reconfiguring.

VERIFIED

The rebuilt image — kernel with `CONFIG_DRM_VIRTIO_GPU=y`, `libdrm`, and Mesa’s `libEGL / libGLv2 / libgbm / libgallium-26.2.2.so` all present in the rootfs — still boots cleanly through the same checks as every earlier milestone: `login: root → -bash-5.2# → dbus-send` returns a real reply → `udevadm info` returns a populated device entry. No regression from Phase 2. Nothing yet **exercises** the new graphics libraries — that’s the rest of Phase 3’s own milestone (a minimal Wayland client rendering via `virtio-gpu`, which needs a compositor to host it, not yet built) — so this is “builds, installs, and boots cleanly,” the same honest bar Phase 2’s Wayland-core libraries table (§6) was held to.

7.1 WESTON, AND THE CAIRO CHAIN THAT BLOCKED IT

Weston is the compositor Phase 3’s milestone needs to host a client — the one piece missing from “a library a compositor can call into” above to “a Wayland client actually renders something.” Building it turned out to need six more from-source packages first, none of them optional: weston’s own `shared/meson.build` calls `dependency('cairo')` and `dependency('libpng')` with no `required: false` — mandatory, not just a demo-client extra — for its kiosk-shell window-decoration code.

PACKAGE	VERSION	WHAT IT’S FOR
zlib	1.3.2	Compression — libpng’s and cairo’s own dependency.
expat	2.7.1	XML parsing — fontconfig’s config-file dependency.
libpng	1.6.44	PNG images — weston’s own unconditional dependency, and cairo’s PNG surface backend.
FreeType	2.13.3	Font rasterization. <code>harfbuzz/bzip2/brotli/png</code> support all disabled — none are built here, and none are needed for cairo’s plain toy-font API.
fontconfig	2.15.0	Font matching — FreeType’s companion, needed for cairo’s font backend to do anything useful. XML config via expat, not libxml2.

PACKAGE	VERSION	WHAT IT'S FOR
cairo	1.18.2	2D graphics — weston's mandatory window-decoration dependency. X11/xcbe/GL backends disabled (image-surface only, no libxcb, §9).
Weston	16.0.0	The compositor itself — DRM backend, GL renderer, kiosk shell, weston-simple-egl only. Vulkan/X11/Xwayland/systemd/JPEG/WebP/LCMS2/docs/tests all disabled.

THE SAME PKG_CONFIG_SYSROOT_DIR LIMITATION, TWICE MORE — AND ONE FIX THAT BACKFIRED

Two more instances of the same PKG_CONFIG_SYSROOT_DIR design limitation from earlier in this section, plus a lesson from trying to close the whole bug class at once instead of one instance at a time:

- **weston's own shared/meson.build** unconditionally probes for pango / pangocairo / fontconfig / glib-2.0 (all required: false, but with no meson option to skip the probe itself) — this build host has apt-installed pango / glib dev packages at genuine standard locations, so the probe “succeeds,” and cairo-util.c ends up #include <pango/pangocairo.h> with no real header at the sysroot-rewritten path the compiler was given. None of pango, glib, harfbuzz, or fridibi are built here, and none are actually needed — weston's kiosk-shell decoration works fine on cairo's plain toy-font API. Fixed the same way as the earlier hwdata case: an idempotent source patch (patch_disable_pango) that forces the probe's own if to never fire.
- **The tempting “real” fix** — set PKG_CONFIG_LIBDIR to only the sysroot's own pkgconfig directories, replacing pkg-config's built-in host search path entirely instead of just prepending to it — closes this whole bug class in one move, and was tried. It broke a different, **legitimate** case instead: wayland-server needs libffi, which — like glibc itself — this project deliberately never builds from source, relying on the host's copy the same way it already does for the runtime .so (§8.2's HOST_DYNAMIC_LIBS list). Excluding the host search path made that real dependency unfindable too. Reverted in favor of the established pattern: an explicit per-package disable each time an **unwanted** leak is actually hit, not a blanket restriction that can't distinguish “unwanted optional leak” from “deliberately host-provided dependency.”

VERIFIED

weston and weston-simple-egl both build and install as genuine dynamically-linked ELF binaries in the sysroot. readelf -d ... | on every new .so in the chain — libweston-16, libexec_weston, drm-backend.so, grep NEEDED

gl-renderer.so, kiosk-shell.so, and cairo/fontconfig/freetype/libpng/expat/zlib themselves — shows zero leftover host-only runtime dependency: everything resolves to either one of these seven packages, an already-built Phase 2/3 sysroot library, or glibc.

Since this table, the actual milestone landed: assemble-rootfs picked up all seven packages automatically with zero code changes (they install into the same real sysroot the old pipeline already bulk-copies), and a real QEMU boot got weston all the way to a running compositor with weston-simple-egl connected and rendering — DRM/EGL/GL initialize using our own Mesa softpipe renderer, libseat grants session control, libinput configures all three input devices, and kiosk-shell.so loads. Getting there past this table surfaced two more real, previously-unnoticed gaps, both fixed and covered next: a missing xkeyboard-config data package, and eudev's own rules directory being silently wrong.

7.2 XKEYBOARD-CONFIG, AND A SECOND PKG_CONFIG_SYSROOT_DIR CASUALTY

Building weston is not the same as running it. The first real QEMU boot attempt failed immediately: xkbcommon: ERROR: failed to add default / failed to create XKB context — the already-documented include path /usr/share/X11/xkb

gap (§9) that xkeyboard-config, the keyboard layout/rules **data** package (no C code, nothing links against it — it's what libxkbcommon looks for at **runtime**), was never built. Adding it as an eighth buildpack fixed this cleanly.

The next boot attempt got further — DRM, EGL, and GL all initialized successfully against our own Mesa build — then failed differently: every input device (Power Button, AT Translated Set 2 keyboard, ImExPS/2 Generic Explorer Mouse) was rejected as “not tagged as supported input device,” and weston aborted with “failed to create compositor backend.” strings on the built udevd binary explained why: this build host has systemd's own udev.pc installed (apt's systemd-dev package), which eudev's configure

auto-detects its rules directory from `—` and `PKG_CONFIG_SYSROOT_DIR` rewrote that variable into a **literal build-machine absolute path** (`.../build-distro/sysroot/usr/lib/x86_64-linux-gnu/udev/rules.d`), baked into `udev` as a compiled-in constant. The same `wayland_scanner` -variable bug class documented earlier in this section — except silent instead of an immediate build failure, since that path genuinely exists **on the build machine**, and only breaks once the rootfs is booted as an independent image where it doesn't exist at all. `build_eudev` now passes `--with-rootlibexecdir=/usr/lib/udev` explicitly, bypassing the `sysroot`-mangled auto-detection.

VERIFIED — THE ACTUAL MILESTONE

With both fixes, a real QEMU boot: `udev` tags every input device correctly (“is tagged by `udev` as: Keyboard”/“Mouse”), `weston` (launched with a minimal `weston.ini` selecting `kiosk-shell.so` — without one, `weston` defaults to `desktop-shell.so`, which was never built) initializes DRM, EGL/GL via our own Mesa softpipe Gallium driver, `libinput`, and the kiosk shell, and opens a Wayland socket. `weston-simple-egl`, run against that socket, connects and runs without error or crash — the actual roadmap milestone: **a minimal Wayland client renders via virtio-gpu inside QEMU**. The new buildpacks and the `eudev` fix needed no CLI/rootfs wiring beyond the existing `assemble-rootfs / make-image` — everything lands in the one shared `sysroot` both the old pipeline and the new buildpacks write into.

08 HOW IT'S ACTUALLY BUILT

8.1 THE CONFIG FILE

`distro.toml` is not one monolithic struct. `distro`'s `Config` composes the pieces that are genuinely identical to `distroless` (`KernelConfig`, `ImageConfig`, `UtilsConfig`, all from `builder-core`) with a `[section]` per package in §4–7 — each just a `version` and a source `url` (plus `util-linux`'s `full` toggle). Since §8.5, `Config` itself no longer knows where any package's source extracts to or builds — that naming-convention knowledge (including the GitLab/GitHub double-prefix archive quirks, and `libdrm`'s literal commit-SHA-suffixed directory name) now lives entirely in each package's own buildpack.

8.2 THE PIPELINE, STAGE BY STAGE

DISTRO'S CLI SURFACE (`DISTRO/SRC/CLI.RS`)

```
distro fetch           # download + extract every source
distro build-toolchain # apt-get the host build tools (once)
distro build-kernel    # the kernel buildpack
distro build-userland   # every other package in §4 through §7
distro assemble-rootfs  # merge it all into build-distro/rootfs
distro make-image       # partition + GRUB + write the disk image
distro test-qemu [--window] # boot it
distro write-usb --device <dev> # dd to real hardware (destructive)
distro all              # the whole pipeline, in order
```

Every one of these now runs through the buildpack registry described in §8.5, not a hand-written per-stage function — `distro/src/stages/`, which used to hold that code, are gone entirely. `{fetch,userland}.rs`

`build-userland` fetches and builds every package but the kernel, in dependency order; `assemble-rootfs` then copies every `StaticArtifacts` package's declared outputs in directly (`coreutils` and its 33 applet symlinks, `bash` + `bin/sh`, `util-linux`'s `agetty` / `mount` / `umount`, `shadow`'s `login` / `passwd`, `distro-init` as `/sbin/init`), bulk-copies the whole shared `sysroot` for every `Sysroot` package in one `cp -a`, then adds the host's own `libc.so.6` / `libexpat.so.1` / `libm.so.6` / `libgcc_s.so.1` / `libstdc++.so.6` / `libz.so.1` / `libzstd.so.1` / `libffi.so.8` (the list has grown package by package — `libffi` turned out to be a latent gap since Phase 2's `libwayland-client`, just never caught until Mesa's EGL exercised it too) and the dynamic linker (confirmed via `ld-linux-x86-64.so.2 --help` to already be on `glibc`'s default search path here — no `ldconfig` step needed), plus `/etc/passwd` + `/etc/shadow`.

8.3 THE SYSROOT: HOW PACKAGES IN §5-7 FIND EACH OTHER

Phase 1's five packages never needed each other at build time — each just needed the host's `gcc`. §5-7's packages do: `wayland-protocols` needs `wayland-scanner` on `PATH` at its own build time, and `libinput` needs `eudev`'s installed `libudev.pc` to link against `libudev` at all. Every package in §5-7 is therefore built with `--prefix=/usr` (its normal, final, “as if genuinely installed” prefix) and installed with `DESTDIR=<build-distro/sysroot>` — files physically land under the sysroot, but the package's own compiled-in idea of its prefix stays `/usr`, which matters: `dbus`, for instance, looks up its own config file relative to whatever prefix it was **actually built with**, at its own runtime, on the real target — not at build time. `PKG_CONFIG_SYSROOT_DIR`, a mechanism `pkg-config` itself provides, then rewrites the `-I / -L` paths a later package's build sees from `/usr/...` to the sysroot's real, on-disk `<sysroot>/usr/...`. `assemble-rootfs` copies that whole sysroot tree into the final rootfs with one `cp -a`.

ONE PACKAGE BREAKS THIS PATTERN, ON PURPOSE

`wayland` alone is built differently: with a real, absolute, on-disk `--prefix=<sysroot>/usr` and installed directly, no `DESTDIR`. `wayland-scanner`'s own path is baked into `wayland`'s `.pc` file as a **custom** variable (`wayland_scanner=${bindir}/wayland-scanner`) that — unlike ordinary `Cflags/Libs` — `PKG_CONFIG_SYSROOT_DIR` does not rewrite, and `wayland-protocols`' build executes whatever path that variable names, directly, as part of its own build. With `--prefix=/usr` that path would read the literal string `/usr/bin/wayland-scanner`, which does not exist anywhere on the host. This is safe specifically for `wayland` because none of its own **shared libraries** do a prefix-derived runtime lookup the way `dbus` / `eudev`'s daemons do.

An earlier version got this backwards — every package used the absolute sysroot prefix, not just `wayland` — and it booted, but `dbus-daemon` crash-looped: `distro-init` logged “`dbus-daemon` exited, respawning” on repeat, because `dbus` had baked in a literal build-machine path (`.../build-distro/sysroot/usr/share/dbus-1/system.conf`) as its config search location. Found by an actual QEMU boot, not by review.

8.4 HOW EVERY CLAIM IN THIS REPORT WAS CHECKED

A Python harness opens a PTY (`pty.openpty()`, with `TIOCSWINSZ` set — QEMU's serial console renders nothing at a 0×0 terminal size), launches `qemu-system-x86_64` with the built image over `virtio` and `OVMF` for UEFI boot, and drives the console with a small state machine keyed on regexes over the accumulated output — `login:`, a shell prompt, a sentinel string echoed after each command — rather than fixed sleeps.

A BUILD-PIPELINE BUG THIS SAME METHOD CAUGHT

`assemble-rootfs` once failed outright with “No such file or directory” copying the freshly built `distro-init` binary. Cause: this build host sets `CARGO_TARGET_DIR` to a shared cache outside the repo, which `cargo` honors — so the binary was never at the hardcoded `target/...` path the rootfs-assembly code assumed. Fixed by reading `CARGO_TARGET_DIR` at the same call site `cargo` itself does.

8.5 A BUILDPACK ARCHITECTURE

Every package used to be defined the same way: a `{version, url}` config struct plus a `build_dir()` method in `Config`, a fetch call in `fetch.rs`, a `build_<name>` function in `userland.rs`, and — for the static packages (§4) — an `install_<name>` function in `rootfs.rs`. That smearing of one package's identity across four files stopped scaling once `distro` passed ~17 packages, so a new foundational crate, `buildpack-core`, defines a `Buildpack` trait instead: one self-contained value per package, declaring its own source, dependencies, build instructions, and build outputs, plus a description for an eventual generic TUI (today only `distroless` has one, hand-maintained separately from its own stage functions).

CRATE	ROLE
<code>buildpack-core</code>	The trait itself, <code>graph::topo_order</code> (a real topological sort over each package's declared dependencies, replacing a hand-maintained call sequence), and the shared meson/autotools/cargo build helpers.
<code>buildpacks</code>	One implementation per package, in one crate regardless of which distro(s) end up using it — which distro consumes a package isn't a meaningful axis to split crates on.

Every one of `distro`'s 25 packages is now a buildpack, and `distro`'s real CLI calls every one of them directly — `distro/src/stages/`, the two files that used to hold this logic, are gone entirely, along with `Config's {fetch,userland}.rs`

~15 per-package `*_build_dir()` methods (that naming-convention knowledge, including the GitLab/GitHub double-prefix archive quirks and libdrm's literal commit-SHA-suffixed directory name, now lives in each buildpack's own `build_dir()`). `stages/buildpacks.rs` is the entire non-kernel pipeline: `all_packages()` lists all 24, `ctx_for()` maps each one's id to a `BuildCtx`, and `fetch_new_packages / build_new_packages / install_static_outputs` do exactly what their names say, each in `topo_order`. The kernel keeps its own dedicated `build-kernel / menu-config / list-features` commands (its `FEATURE_PACKS`, §3.2, stay its own internal mechanism — they have no source or build step of their own to be buildpacks about), calling the kernel buildpack directly.

Cutting the **first** three packages over (kernel, util-linux, Mesa) needed two real design fixes, both still load-bearing now that all 25 packages are migrated:

- `topo_order` used to hard-error on any declared dependency id not present in the registry it was given. Mesa's **real** dependencies (libdrm, wayland, libxkbcommon, pixman, libdisplay_info, libinput) weren't buildpacks yet at that point — building a deliberate subset of a larger pipeline is the normal case, not a registration bug — so an unregistered dependency id is silently treated as already-satisfied rather than an error. Once every package migrated, this stopped being a “does it happen to be registered” question and became real: Mesa's and Weston's `dependencies()` now list their full actual dependency sets, and `topo_order` genuinely orders all 25 packages correctly.
- Every cutover package gets a `BuildCtx` pointed at its OLD on-disk build location (`build_dir/<name>`, not the shared `build_dir/sources` the 8 cairo-chain packages use, which never had an “old location” to match) — so already-built trees on disk are recognized as-is instead of the migration triggering a redundant rebuild. A kernel rebuild in particular is far too expensive to redo needlessly, and `utils` needed the special case of `sources_dir = build_dir` itself (its old `utils_build_dir()` had no version-numbered subdirectory, just a bare `build_dir/utils`).

`rootfs.rs`'s five hardcoded `install_coreutils / install_bash / install_util_linux / install_shadow / install_init` functions are now one generic `install_static_outputs`: it iterates every `StaticArtifacts` buildpack and copies each declared output plus its symlinks in (`utils`' 33 applet symlinks moved from a `rootfs.rs` constant into the `Utils` buildpack's own `outputs()`, alongside `bash`'s `bin/sh` and `shadow`'s `login / passwd`). `Sysroot`-mode packages needed no `rootfs`-side change at all, cutover or not: `install_sysroot`'s `cp -a` of the whole shared `sysroot` already picks up whatever any of them left there, regardless of which code built it — exactly how Weston ended up actually running inside QEMU (§7.1) before its own cutover even happened.

VERIFIED

`distro fetch / build-kernel / build-userland` all report every one of the 25 packages “already exists” on a rebuild — zero wasted work. `assemble-rootfs` installs every static binary and its symlinks correctly. A full QEMU regression boot (`login`, `dbus-send`, `udevadm`) and a Weston sanity check (DRM/EGL/GL still initialize via our own Mesa build) both pass exactly as before the migration — the whole pipeline was rebuilt without rebuilding a single package.

`distroless` has since migrated too, reusing the same `buildpack-core / buildpacks` crates rather than a second implementation. Its kernel and `utils` are the literal same buildpacks `distro` uses — `utils` gained a `UtilsVariant (Glibc / Musl)` field since the two distros need genuinely different target triples, cargo feature sets, and static-linking mechanisms, but everything else about the package (source, patch,

applet list, install shape) is identical, so it stayed one buildpack rather than splitting into two. Busybox is new: `allnoconfig`, then a curated applet list patched directly into the generated `.config` text, then `make CC=<musl-gcc with -idirafter fallbacks>` — `CC` has to be a make-time argument rather than an environment variable, since BusyBox’s own Makefile unconditionally overwrites it. `distroless/src/pipeline.rs` mirrors `distro/src/stages/buildpacks.rs` at its own (much smaller) scale; `distroless/src/rootfs.rs` keeps the genuinely distro-specific, non-buildpack assembly logic (`inittab/passwd/fstab`, the `udhcpd` script, kernel module install) that has no `distro` equivalent. The migration also surfaced a real, previously-unnoticed gap: `distroless`’s `utils` had never had the `AT_EXECFN` empty-fallback patch applied — the old `builder-core` pipeline had no patch mechanism at all — unlike `distro`’s `utils`, which always carried it. Reusing the shared buildpack fixed this for `distroless` for the first time. `distroless`’s TUI needed zero changes: it re-execs the same CLI subcommands and its stage output-path checks already point at the same on-disk locations the migrated buildpacks’ `ctx_for` overrides target.

VERIFIED

A scripted QEMU boot of the migrated `distroless` reaches its login prompt, logs in via `busybox ash`, and reports `ls /bin | wc -l` as 44 and a correct `uname -a` — the full rebuild-free pipeline works end to end, same as `distro`’s own cutover above.

Every real build run (for either `distro`) now also writes a Graphviz SVG of the buildpack dependency graph (`dependency-graph.svg`, next to the rest of that distro’s build output) via `buildpack_core::graph::write_svg` — one node per buildpack, one edge per declared dependency, generated fresh on every `build-userland` run rather than by request. A buildpack with more than one output (`shadow`’s `login/passwd` binaries, `weston`’s three) gets each output’s description as an extra label line rather than collapsing to just the package name; `util-linux`’s outputs carry the real scanned binary name (`agetty`, `mount`, `umount`, or every binary `full` mode finds) instead of a generic “util-linux binary” placeholder, which needed `BuildOutput.description` to move from `&'static str` to an owned `String` so it could be built at runtime instead of picked from a fixed literal.

A REGRESSION FROM TRYING TO FIX A BUG CLASS, NOT AN INSTANCE

`sysroot_env` briefly set `PKG_CONFIG_LIBDIR` (which replaces `pkg-config`’s own default search path, rather than just prepending to it) to close the whole “unwanted host package leaks into a sysroot build” bug class in one move. It broke a real case instead — `wayland-server`’s legitimate, deliberately-host-provided `libffi` dependency became unfindable — and was reverted in favor of the established pattern: an explicit per-package fix each time an actual unwanted leak is hit, not a blanket restriction that can’t tell “leak” from “genuine host dependency” apart. See the `weston/cairo` section above for the case that prompted trying it.

8.6 FINISHING THE ROADMAP: ONE CONFIG, ONE PIPELINE TRAIT, ONE TUI

The buildpack migration’s own follow-up list named five remaining items. All five are done, in dependency order, each verified with a real QEMU boot before moving to the next:

1. **A shared `DistroConfig`.** `distro/src/config.rs` used to hand-declare ~15 config structs (`BashConfig`, `UtilLinuxConfig`, `MesaConfig`, ...) — exact duplicates of shapes each buildpack already owned privately — purely so its own `Config::load/save` could round-trip them. `buildpack_core::config::DistroConfig` replaces it (and `builder_core::config::Config`) with three typed fields (`build_dir`, `networking`, `image`) plus `packages: BTreeMap<String, toml::Value>` — every other TOML section kept raw and handed to that buildpack’s own `configure()`. A new `Buildpack::to_toml()`, the mirror of `configure`, lets `DistroConfig::save` round-trip whatever changed a buildpack’s config (`resolve-kernel` writing a new version/url; the TUI’s settings screen toggling a kernel feature) without needing a whole-`Config` round-trip through typed fields that no longer exist.
2. **A `PipelineStage` trait** (`buildpack_core::pipeline`) for the three operations that act on the whole assembled distro rather than any one package — `make-image`, `test-qemu`, `write-usb` — ported unchanged from `builder-core`. Toolchain setup turned out **not** to be shared logic (native `apt gcc/meson/ninja` for `distro` vs. `musl-tools+rustup` for `distroless`), so it stayed a distro-specific function

in each `main.rs` rather than becoming a fourth `PipelineStage`; assembling the rootfs stayed distro-specific for the same reason (genuinely different init systems).

3. **builder-core retired entirely.** Everything it held was already superseded — `fetch / userland / rootfs` by the buildpack cutover, `kernel`’s build logic by `buildpacks::kernel`, `image / qemu / usb` by `PipelineStage`. The one real blocker was `distroless`’s TUI, still reading `builder_core::config::Config` and `builder_core::stages::{usb::Device, kernel::FEATURE_PACKS}` directly — repointing those (`kernel` feature toggles now go through `DistroConfig`’s untyped `packages["kernel"]` table) came first, then the crate was deleted outright.
4. **list-packages / fetch-pkg / build-pkg / clean-pkg** CLI subcommands on both distros — natural once a real `Vec<Box<dyn Buildpack>>` registry existed, and the thing Phase 4/5 buildpacks (the Rust toolchain, COSMIC components) will use to be built and debugged one at a time as they’re added.
5. **A generic TUI.** `distroless/src/tui/{app,mod,stage}.rs` lifted into their own crate, `builder-tui`, almost unchanged — the `re-exec / event-loop / settings` machinery was already distro-agnostic once both distros shared `DistroConfig` and `buildpacks::kernel`. The one real seam, each distro’s own package set, is injected via a `Registry` trait (`all_packages / ctx_for / kernel_buildpack / kernel_ctx / pipeline_ctx / rootfs_ready_marker`) that `distro` and `distroless` each implement as a thin delegation to their own `stages::buildpacks / pipeline` module. Stage presence/clean for `Fetch / BuildKernel / BuildUserland` now calls the real `Buildpack::is_built / clean` through the `Registry`, instead of hand-computing marker paths against a config shape that no longer exists — the actual generalization the original plan asked for, not just a code move. `distro` gets a working `Tui` command for the first time in the process (net-new capability).

VERIFIED

Every phase above got its own `cargo build --workspace` plus a real QEMU boot (not just a compile check) before the next one started. The final state: both distros’ full `fetch` → `build-userland` → `assemble-rootfs` → `make-image` pipelines report every package “already exists” with zero rebuilds, both boot cleanly in QEMU, and the new `distro tui / distroless tui` dashboards render correct build-status markers and each distro’s own `hostname/networking / kernel-feature` settings.

With this done, Phase 4 (a Rust toolchain) and Phase 5 (COSMIC components, §9) need no new architecture — each is just another entry in `all_packages()`, built and iterated on via `build-pkg <id>` like anything else here.

09

WHAT ISN'T PART OF THE PICTURE YET

- PHASE 3 **Done** — §7.1’s actual milestone verified in a real QEMU boot: `weston-simple-egl` connects to a running Weston compositor and renders via `virtio-gpu/Mesa softpipe`. Not yet done, and left for later: making this happen automatically at boot (today it’s started by hand from a login shell with a hand-written `weston.ini`, not a `distro-init` service) — a smaller, well-understood remaining step, not a new unknown.
- PHASE 4 `rustup / cargo on-target`, plus a curated Rust-CLI-tools suite (`ripgrep`, `bat`, `eza`, ...).
- PHASE 5 COSMIC itself — `cosmic-comp`, `cosmic-session`, `cosmic-panel`, `cosmic-greeter`, minimal subset first.
- PHASE 6 Expand: more COSMIC components, real GPU drivers beyond `virtio-gpu`, audio, networking UI.

And three deliberate gaps in what’s already built, worth knowing about rather than discovering later:

- `utils`’ built feature set does not include `grep` or `sed`, despite both names appearing in the `Utils` buildpack’s `applet-symlink` list — they were never part of `coreutils`’ scope upstream in the first place. The symlinks exist but dangle. Not yet fixed.
- `libxcb` was never built. Its only real consumer on a Wayland-native target is XWayland compatibility, not currently planned — three more from-source packages (`libxcb`, `libXau`, `libXdmcp`) not worth building for a checklist item with no near-term user.
- `mtdev` (legacy multitouch) and `libwacom` (tablet identification) were both left out of `libinput` — niche hardware support, easy to add back later if a real device needs it.